Deep Learning - leveling up

Maciej Kępa

Roadmap

Data Community

01

AI/ML architecture - how it all fits together

Understand the full ML stack from data sources to production, including roles, cloud reference architectures, layers, tools, and a simple ETL + model demo.

02

Data Preparation - practical foundations

Learn practical techniques for building ML data pipelines, cleaning data, validating datasets, and preparing a dataset in PySpark

03

Feature Engineering – the art of extracting value from data

Discover how to create effective features, handle different types of data, apply encoding and normalization, monitor feature drift, and track features with Spark + MLflow.

04

ML Algorithms - the classical approach

Get familiar with core algorithms like regression, trees, and gradient boosting, understand classification, regression, clustering, their pros and cons, and try MLlib demos in Databricks.

05

Model Training in Practice

Learn how to split data, perform hyperparameter tuning, scale training, and train models with code and metrics using MLflow.

06

Deep Learning - leveling up

Understand neural network basics, choose between TensorFlow and PyTorch, leverage GPU/TPU scaling, and build a simple CNN with visualization.

07

ML Pipelines - automation and CI/CD

Build repeatable ML workflows with orchestration tools, CI/CD pipelines, model registry/versioning, and a Databricks pipeline demo.

08

MLOps - manage your ML solution

Explore ML lifecycle management with tracking, model monitoring, data drift handling, canary deployments, A/B testing, and monitoring demos.

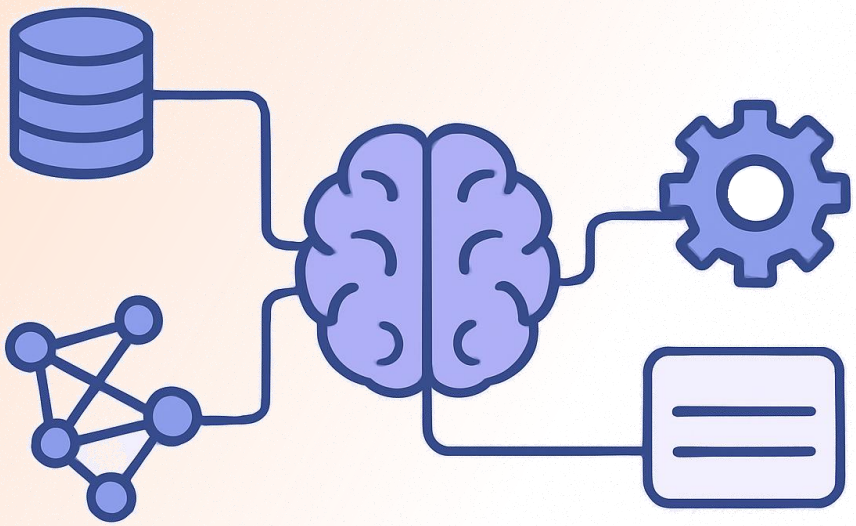
09

Generative AI and LLMs - the new wave of technology

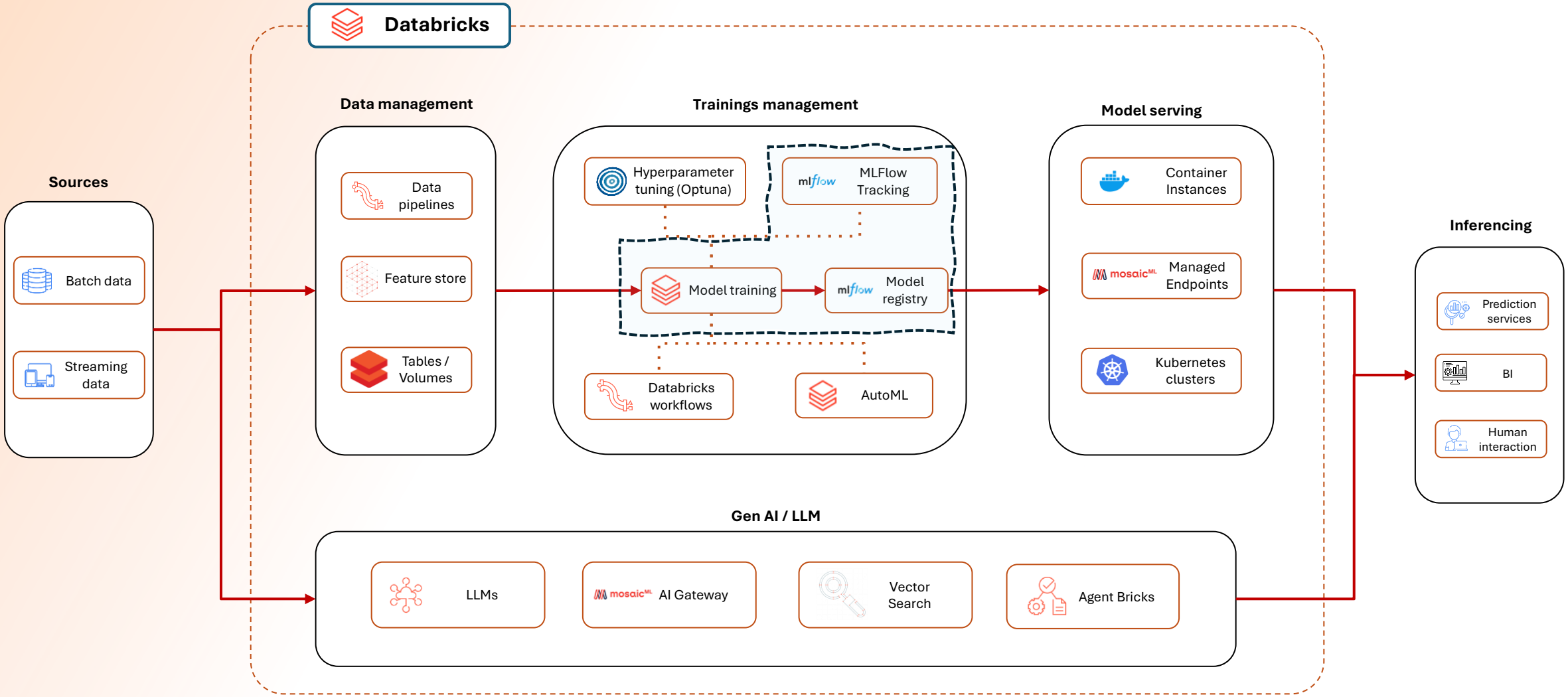
Learn how LLMs differ from classical ML, understand transformers, RAG, AI agent orchestration, and try a simple RAG demo in Azure OpenAI.

Agenda

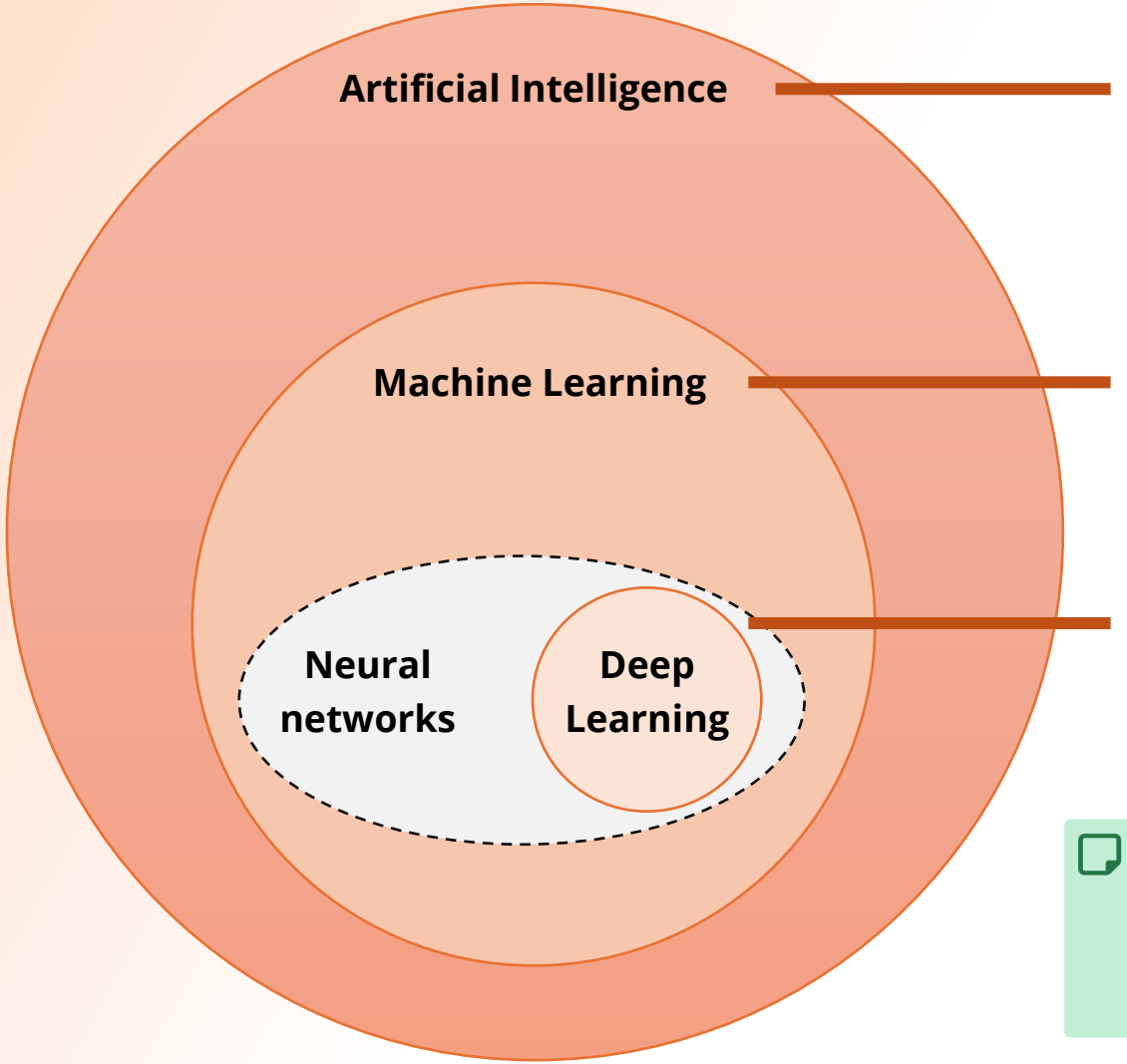
1. Introduction
2. Neural network basics
3. Deep learning training and frameworks
4. Workshop: multi-layer perceptron
5. Workshop: convolutional neural network
6. Workshop: transfer learning
7. Workshop: autoencoders
8. Best practices



Introduction


Databricks
Sources
Batch data
Streaming data
Data management
Data pipelines
Feature store
Tables / Volumes
Trainings management
Hyperparameter tuning (Optuna)
MLFlow Tracking
Model training
Model registry
Databricks workflows
AutoML
Model serving
Container Instances
Managed Endpoints
Kubernetes clusters
Inferencing
Prediction services
BI
Human interaction
Gen AI / LLM
LLMs
AI Gateway
Vector Search
Agent Bricks

The AI Ecosystem

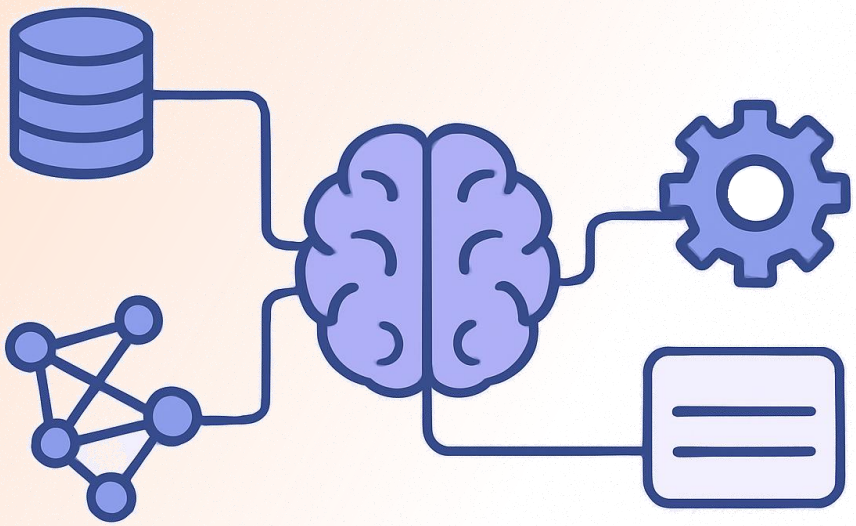


Artificial Intelligence represents the broadest category, encompassing all systems that exhibit human-like intelligence. This includes rule-based expert systems, optimization algorithms, and learning systems.

Machine Learning is a specific approach to achieving artificial intelligence through data-driven learning rather than explicit programming.

Deep Learning represents a specialized ML techniques that use multi-layered neural networks to automatically learn from complex, unstructured data.

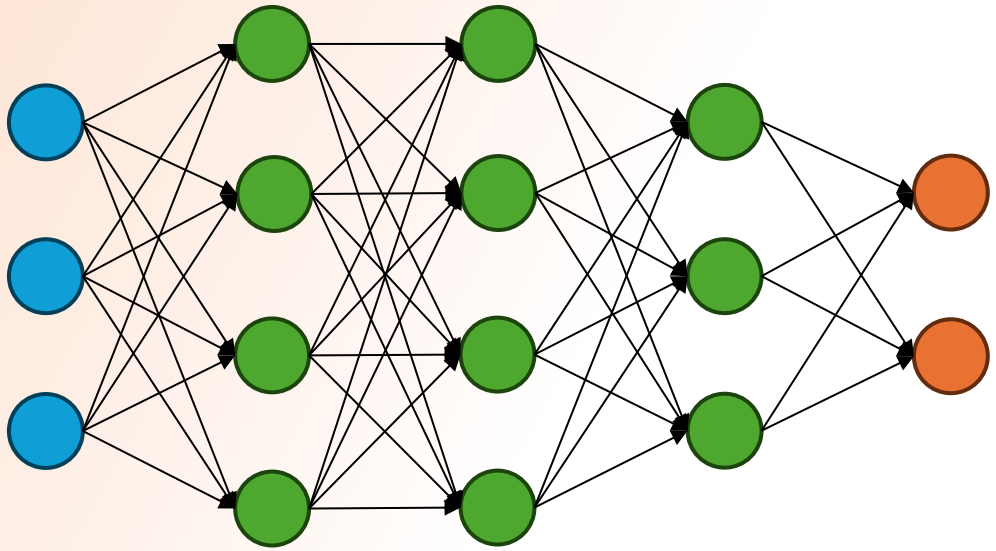
⚠️ All deep learning is machine learning, but not all machine learning is deep learning. The use of multi-layered neural networks is what typically distinguishes DL from ML, but there is no 'magic number' of layers that marks the transition



Neural network basics

The blueprint - neural network

Deep neural networks consist of interconnected layers of processing units called **neurons**. Each layer transforms input data through weighted connections and nonlinear activation functions. The architecture typically comprises three distinct types of layers working together to learn complex patterns from data.



01

Input Layer

Receives raw data features and passes them to the first hidden layer. Each neuron represents a feature dimension.

02

Hidden Layers

Multiple intermediate layers that learn increasingly abstract representations through weighted connections and nonlinear transformations.

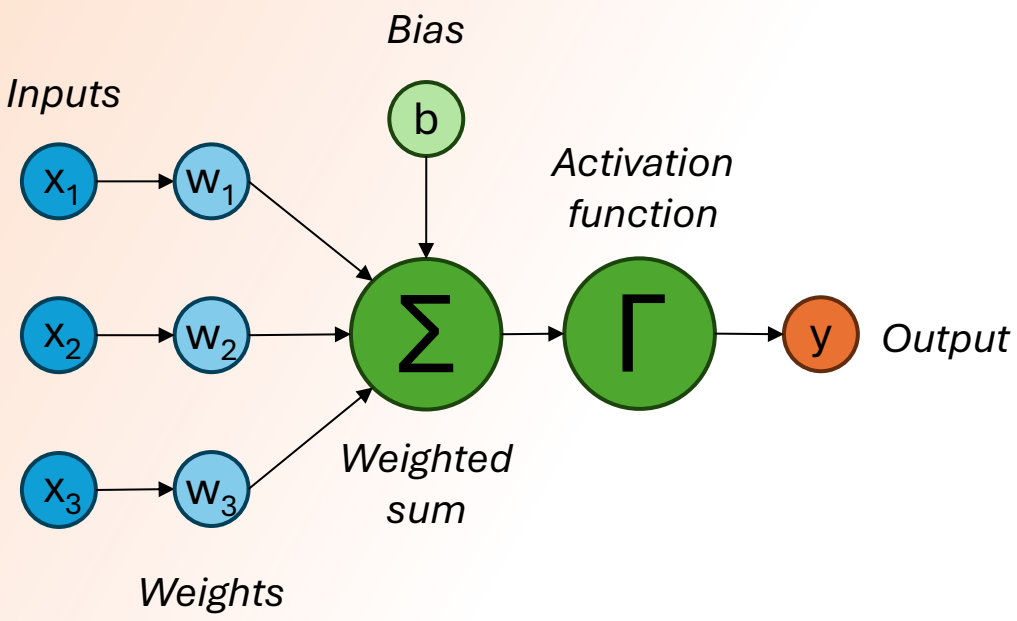
03

Output Layer

Produces final predictions using appropriate activation functions tailored to the task (classification, regression, etc.).

The building block - neuron

A single processing unit (**neuron**) mimics the core function of its biological counterpart. It takes multiple inputs and performs specific mathematical operations to produce an output, forming the foundational component of any neural network.



01

Inputs & weights

Raw values (x) are received and multiplied by trainable weights (w) that dictate their importance.

02

Weighted sum & bias

All weighted inputs are summed to aggregate the signals. A learnable bias term is added to the summation to shift the activation threshold.

03

Activation function

A mathematical non-linearity (like ReLU or Sigmoid) is applied to the final sum.

04

Neuron output

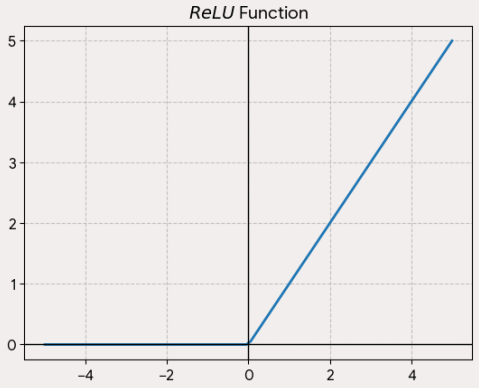
The final transformed value is produced, ready to be passed to the next layer of neurons in the architecture.

The valves - activation functions

Activation functions introduce non-linearity into neural networks, enabling them to learn complex patterns and decision boundaries. Each function serves specific purposes depending on the network architecture and task requirements.

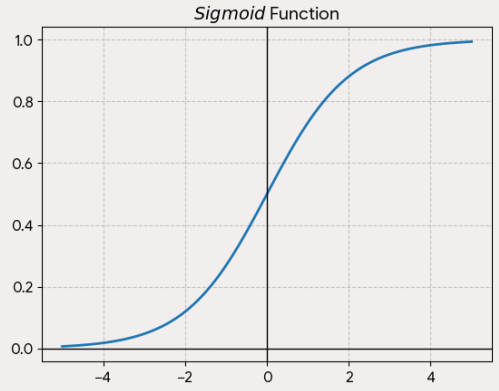
ReLU (Rectified Linear Unit)

Most common activation for hidden layers. Computationally efficient with fast convergence. Outputs max(0, x), introducing sparsity and biological plausibility.



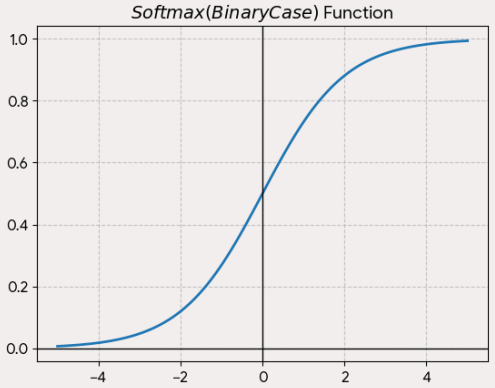
Sigmoid

Maps inputs to (0, 1) range. Ideal for **binary classification** output layers. Prone to vanishing gradients in deep networks.



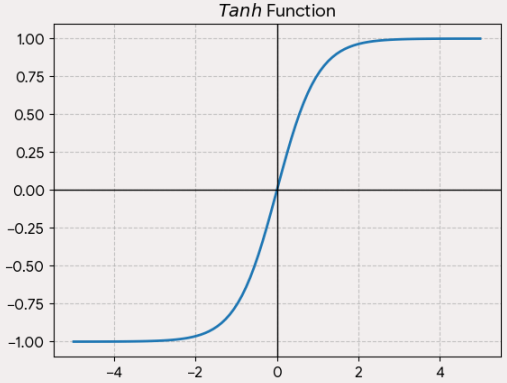
Softmax

Converts outputs to probability distribution for **multi-class classification**. Ensures outputs sum to 1, representing class probabilities.



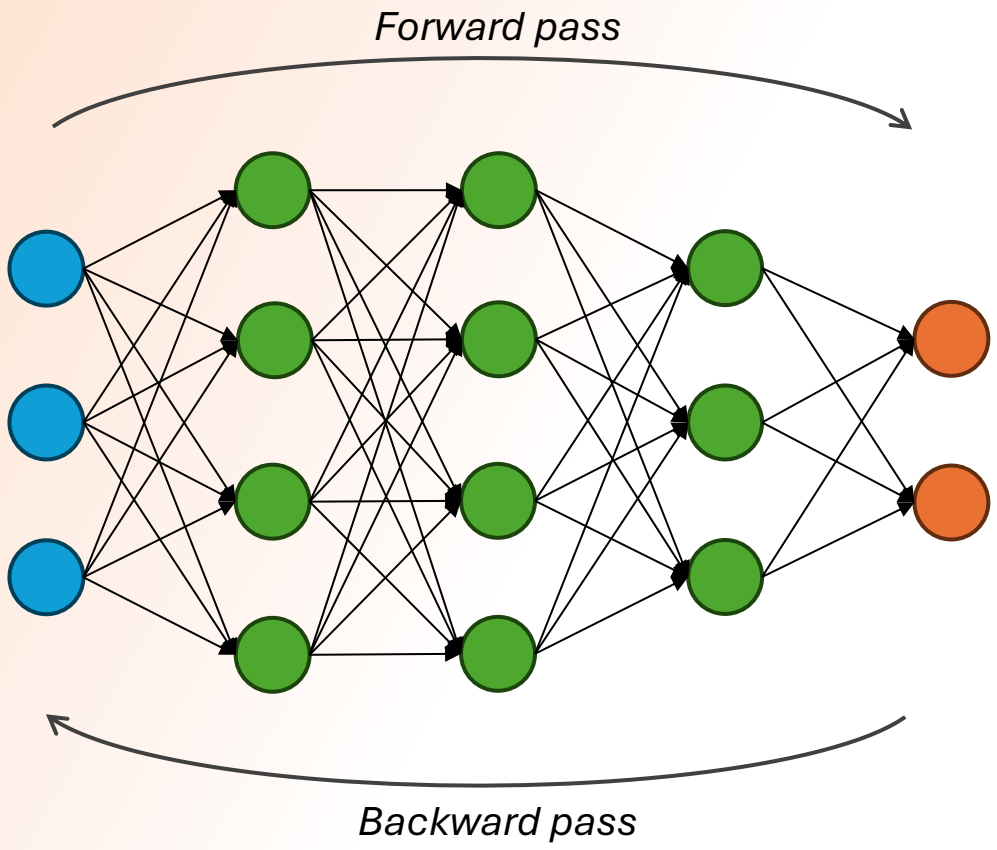
Tanh

Hyperbolic tangent mapping to (-1, 1). Zero-centered outputs aid optimization. Alternative to sigmoid for hidden layers.



The engine - backpropagation

Backpropagation applies the chain rule from calculus to compute how each weight contributes to the final error. This "blame assignment" tells us exactly which weights need adjustment and by how much. The beauty lies in efficiency: computing gradients for all weights simultaneously requires similar scale computational cost as a single forward pass, making deep networks trainable.



1

Forward Pass

Compute predictions by propagating inputs through the network, calculating activations at each layer.

2

Loss Calculation

Measure discrepancy between predictions and ground-truth using loss functions (MSE, cross-entropy, etc.).

3

Backward Pass

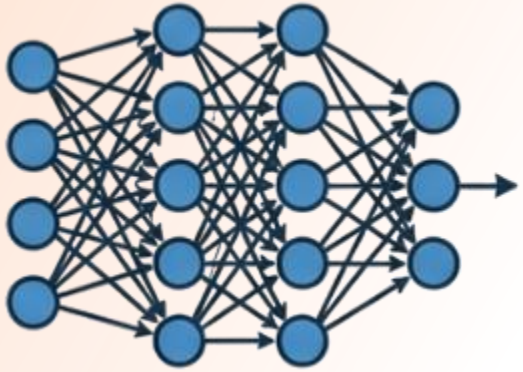
Propagate error gradients backward through layers, applying chain rule to compute parameter gradients.

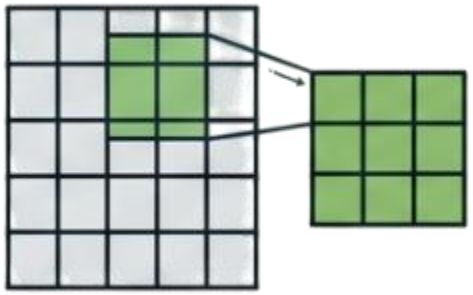
4

Parameter Update

Adjust weights and biases using gradient descent to minimize loss function iteratively.

The assembly stages - layers





Dense Layer

How it works: Every neuron connects to every neuron in the previous layer, performing weighted sums of inputs followed by non-linear activation functions

Use Cases: Classification and regression tasks, final layers after feature extraction, universal function approximation

Trade-offs: High parameter count makes it computationally expensive and prone to overfitting with large input sizes

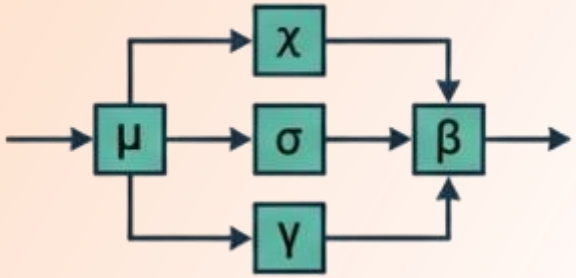
Convolutional Layer

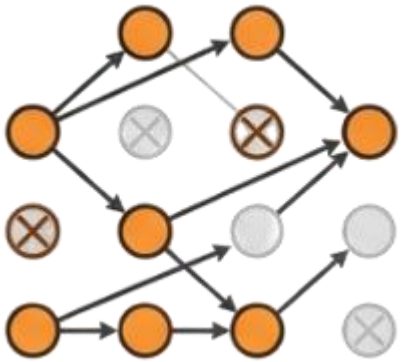
How it works: Uses filters (kernels) that slide across input data to detect spatial hierarchies of features - from edges and textures to complex objects. Exploits spatial locality through parameter sharing

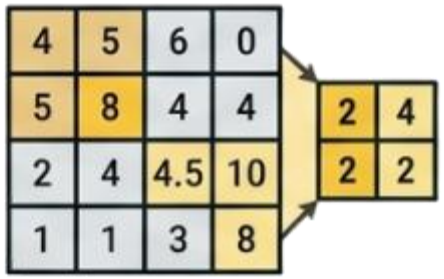
Use Cases: Image recognition, computer vision, video analysis, and any task involving spatial data with local correlations

Trade-offs: Less effective for non-spatial data; requires careful tuning of filter size, stride, and other hyperparameters

The assembly stages - layers







Batch Normalization Layer

How it works: Normalizes the activations of a layer by adjusting the mean and variance for each mini-batch, then scales and shifts the result using learned parameters

Use Cases: Very deep architectures where training is slow or unstable

Trade-offs: Adds overhead to the training process and depends heavily on batch size (performs poorly with very small batches).

Dropout Layer

How it works: Randomly "deactivates" a percentage of neurons during each training step, forcing the network to learn redundant representations

Use Cases: Regularization for large models to prevent overfitting

Trade-offs: Increases training time; must be disabled during inference (testing), which can cause scaling issues if not implemented correctly

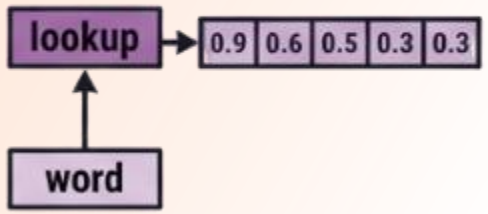
Pooling Layer

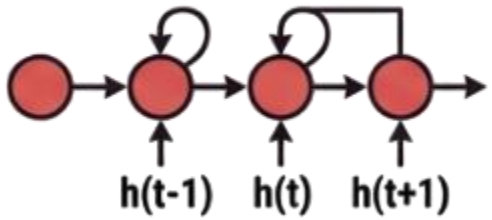
How it works: Reduces the spatial dimensions of an input by taking the maximum or average value from a sub-region (e.g., 2×2 window)

Use Cases: Downsampling feature maps, reducing computational load, and providing translation invariance

Trade-offs: Information loss - discarding the exact location of features can be detrimental in tasks like image segmentation

The assembly stages - layers





Embedding Layer

How it works: Maps high-dimensional, discrete items (like words or IDs) into a low-dimensional, continuous vector space where similar items are closer together

Use Cases: Natural Language Processing (NLP), recommendation systems, and handling high-cardinality categorical data

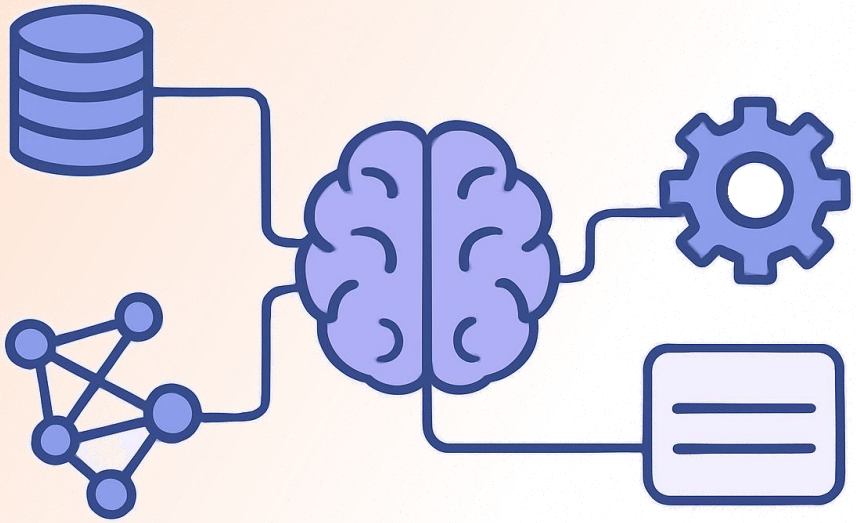
Trade-offs: Can consume massive amounts of memory for large vocabularies; requires a lot of data to learn meaningful relationships

Recurrent Layer

How it works: Uses a feedback loop where the output of the current step is fed back into the next step, maintaining a "hidden state" or memory. Processes data in sequence

Use Cases: Time-series forecasting, speech recognition, and language translation

Trade-offs: Difficulty learning long-term dependencies (vanishing gradients) and lacks the parallelization efficiency of newer models like Transformers



Deep learning training and frameworks

The control room - frameworks

Two dominant deep learning frameworks power modern ML development. While offering comparable capabilities, they differ fundamentally in design philosophy, execution model, and ecosystem.

TensorFlow (Google)

Strengths: Production-grade ecosystem, TensorBoard visualization, TensorFlow Serving for deployment, historical static computational graphs

Use Cases: Scalable industrial deployments, production inference pipelines, enterprise applications requiring stability

Ecosystem: Extensive tooling for model serving, monitoring, and deployment across platforms

PyTorch (Meta)

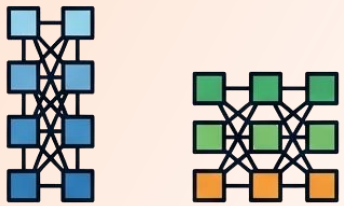
Strengths: Dynamic computational graphs (eager execution), intuitive imperative programming, rapid prototyping speed, Pythonic interface

Use Cases: Research, academic projects, fast experimentation, development requiring flexibility

Ecosystem: Strong research community, TorchScript for production, seamless integration with Python tools

The regulators - architecture

Every architectural decision - from layer count to activation functions - shapes how well your model will learn your data patterns.

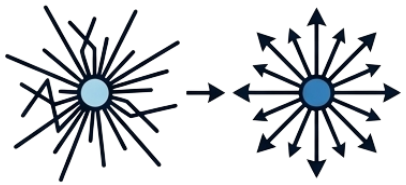


Layer depth vs. width

Deeper networks excel at learning hierarchical features and complex patterns. Wider networks increase memory capacity and parallel processing ability. The choice depends on your data complexity and computational budget.

Activations & Layer types

Activation functions enable the model to learn complex decision boundaries beyond simple linear trends. Specialized layer types adapt the architecture to specific data structures. Choosing the right 'gate' and 'tool' ensures efficient feature extraction.

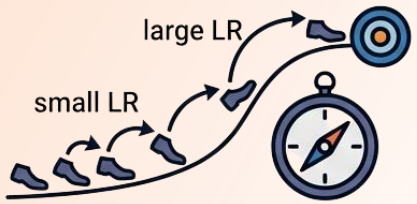


Weight Initialization

Proper weight initialization prevents the "dying network" problem where gradients vanish before learning begins. Modern techniques like Xavier or He set initial weights to maintain stable gradient flow throughout training.

The regulators - hyperparameters

Hyperparameters control the training process itself. Unlike weights learned from data, these must be set before training begins. Finding optimal values often determines success versus failure.



Learning rate

Most critical parameter. Controls step size during weight updates. Too high causes divergence and oscillation. Too low results in painfully slow convergence. Adaptive methods (Adam, RMSprop) automatically adjust learning rates.

Batch size

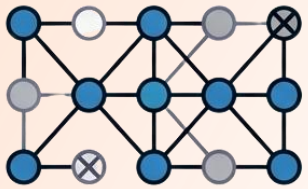
Number of samples processed before updating weights. Smaller batches introduce gradient noise that can help escape local minima but increase training time. Larger batches smooth gradients but require more memory.

Epochs & early stopping

Number of complete passes through training data. Too few epochs underfit, too many overfit. Early stopping monitors validation loss to halt training when performance plateaus, preventing overfitting while saving time.

The regulators - regularization

Overfitting occurs when models memorize training data instead of learning generalizable patterns. Regularization techniques force models to learn robust features that work on unseen data.

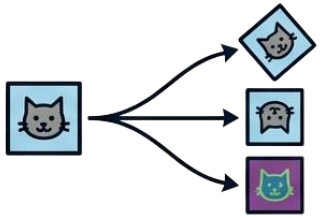


Dropout

Randomly "disables" neurons during training, forcing the network to distribute learning across multiple pathways. At test time, all neurons activate with scaled weights. Makes networks more robust to individual neuron failures.

L1/L2 Regularization

Adds penalty terms to loss function that discourage large weights. L2 (ridge) shrinks all weights smoothly. L1 (lasso) drives some weights to exactly zero, performing feature selection. Prevents over-reliance on specific features.



Data augumentation

Artificially expands training data through transformations (rotations, flips, crops). Provides more diverse examples without collecting new data. Particularly powerful in computer vision where transformations preserve semantic meaning.

The vision - monitoring & metrics

Effective training requires constant vigilance through comprehensive monitoring. Beyond simple accuracy metrics, practitioners must track loss curves, gradient magnitudes, and hyperparameter configurations across experiments.

01

Track Loss Curves

Monitor training and validation loss to identify overfitting, underfitting, or optimization instability. Sudden spikes indicate exploding gradients.

02

Analyze Precision & Recall

Accuracy alone masks class imbalance issues. Precision-Recall curves reveal true model performance across different thresholds and class distributions.

03

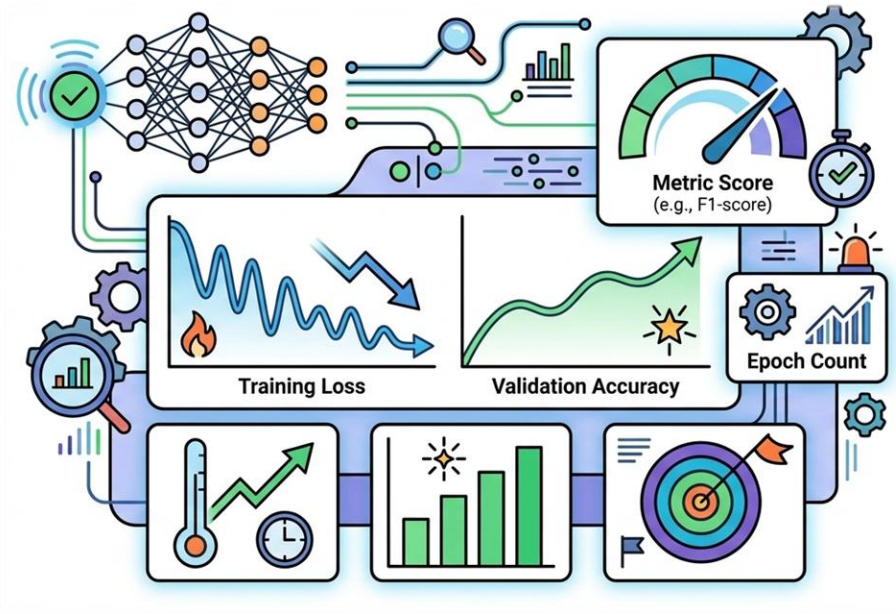
Log Everything

Record hyperparameters, random seeds, and environment details for reproducibility. Experiment tracking systems (MLflow, Weights & Biases) automate this process.

04

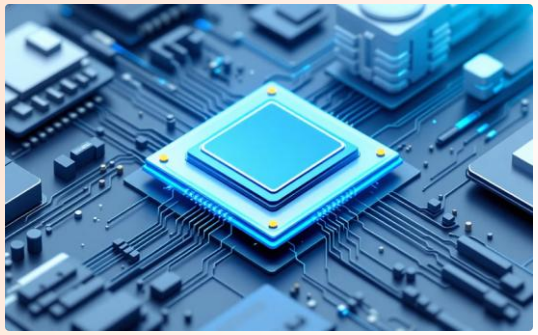
Visualize on board

Interactive dashboards enable real-time monitoring of metrics, computational graphs, and embedding projections during training.



The power plant - hardware

Computational hardware fundamentally determines training time and model scale. Each processor type serves distinct roles in the deep learning workflow, from prototyping to production inference.



Central Processing Unit

Excellent for sequential logic, small models, and inference on edge devices. Limited parallelism makes CPUs impractical for large-scale training.

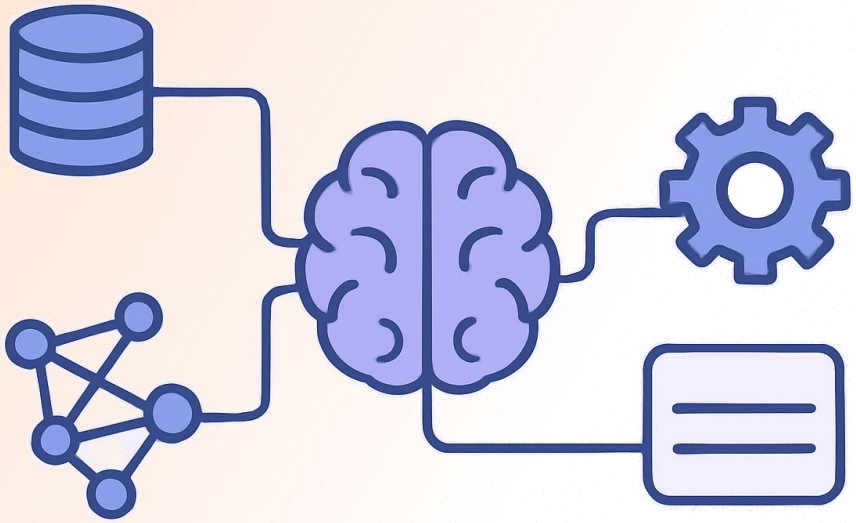
Graphics Processing Unit

Industry standard for training with thousands of cores optimized for parallel matrix multiplication. NVIDIA's CUDA ecosystem provides mature tooling.

Tensor Processing Unit

Google's specialized ASICs designed exclusively for massive tensor operations. Offers superior performance per watt for specific workloads.

DEMO



Summary

Takeaways

Architecture is a Tool, not a Goal: for most real-world tasks, Transfer Learning is the gold standard. Instead of training a 50-layer network from zero, take a pre-trained model like ResNet, freeze the base, and fine-tune it for your specific data. It saves time, energy, and expensive compute.

Monitor Early, Fail Fast: Learning Rate and Loss Curves are the pulse of your model. Use tools like TensorBoard or MLflow to catch Overfitting early. If your training loss drops but validation loss rises, your model is „memorizing" instead of „learning" - stop the training and add regularization.

Data is the "Code" of Deep Learning: in traditional software, logic is in the code. In Neural Networks, the logic is extracted from the data. Whether you are using a simple MLP for tabular data or a CNN for images, the quality of your input determines the ceiling of your model's performance.

Thank you!

Contact:

https://www.linkedin.com/in/maciej-kepa

https://github.com/maciejkepa/ai-ml-in-practice